Laboration 5 – Sorteringslagoritmer

Viktigt! *Som vanligt, läs "Information och regler kring laborationerna".*

Din lösning ska lämnas in i Canvas, gör inlämningen så snart du känner dig färdig med den. Du ska också lämna in en screenshot där din statistik framgår.

Läs varje deluppgift noggrant då det är viktigt att du följer instruktionerna för att du ska få ihop allt rätt. Om du undrar över något eller tycker att något är otydligt, hör av dig och fråga.

Vid inmatning från användaren kan du anta att korrekt typ matas in.

Se till att vara förberedd inför labbtillfällena, så att du verkligen hinner få hjälp med det du eventuellt har problem med.

Tänk igen om hur du ska ta dig an uppgiften och planera lösningen innan du börjar koda – vad är funktionens uppgift och vad måste hända rent logiskt. Rita gärna upp olika scenarion för att förstå vilka problem som kan uppstå.

Uppgift 5.1: Ladda ner och kompilera labbskelettet

Hämta filerna i den zippade filen SortinAlgorithms.zip som finns i Modul F i Canvas. Öppna ett nytt projekt i din utvecklingsmiljö och lägg till filerna i projektet. Programmet ska kompilera korrekt (med eventuella varningar). Det ska också gå att köra utan problem.

Detta labskelett är egentligen mer än ett skelett: det är ett helt program som sorterar och skriver ut resultat av en mängd sorteringar. Du behöver inte sätta dig in i hur det fungerar (om du inte vill, men det kan vara lärorikt), du kommer bara att behöva bry dig om ett litet antal funktioner som beskrivs nedan.

Uppgift 5.2: Bestäm vilka sorteringslagoritmer du vill implementera

I kursen pratar vi om fem olika sorteringsalgoritmer:

- Bubblesort (optimerad)
- Insertionsort (Insättningssortering)
- Selectionsort (Urvalssortering)
- Mergesort
- Quicksort

Du ska implementera ***minst tre av dessa*** (det är naturligtvis tillåtet att implementera fyra eller alla fem också), **en av dem måste dock vara Mergesort eller Quicksort.**

Uppgift 5.3: Implementera sökalgoritmer

Det finns tomma funktionskroppar för samtliga sorteringsalgoritmer i filen `SortingAlgorithms.c`. Samtliga funktioner tar tre argument:

- arrayToSort: arrayen som ska sorteras. Detta är en array av **unsigned int**s (men typedefad till ElementType).
- size: storleken på datamänden som ska sorteras.
- statistics: detta är en pekare till en typ som ska hålla reda på statistik för antalet jämförelser och byten för era algoritmer. Du behöver inte känna till hur typen ser ut för att använda den.

Implementera dina sökalgoritmer. Det är naturligtvis tillåtet att lägga till statiska hjälpfunktioner. För de mer avancerade algoritmerna som mergesort och quicksort kommer det att vara nödvändigt att göra detta eftersom de består av flera steg och är rekursiva.

Statistik

För att statistiken ska fungera korrekt behöver du använda några funktioner som finns i labskelettet. I dina algoritmer ska du aldrig använda jämförelseoperatorer (t.ex <, > , ==, !=) utan istället använda funktionerna `lessThan()`, `greaterThan()`, `equalTo()` osv som finns deklarerade i `statistics.h`. Dessa funktioner returnerar 1 (sant) om jämförelsen stämmer och 0 (falskt) annars. De tar en pekare till statistikvariabeln som argument och uppdaterar den. Notera att du ska använda dessa även i loopar. Så en loop som loopar 10 ggr kommer att se ut t.ex:

```
for(i=0; lessThan(i, 10, statistics); i++)
```

På samma sätt ska du använda funktionen `swapElements()` (också definierad i `statistics.h`) för att byta plats på två element.

Tycker du det är svårt att skriva logiken med dessa jämförelsefunktioner kan du börja med att skriva den med jämförelseoperatorerna men glöm då inte att byta ut samtliga dessa till funktionerna när logiken är färdig.

Om du har gjort rätt kommer programmet att hålla reda på hur många jämförelser och byten dina algoritmer gör.

För att programmet ska fungera

När du vill testa att din sorteringsfunktion fungerar ordentligt så måste du berätta för programmet att sorteringsfunktionen finns, det gör du genom att avkommentera rätt rad i funktionen `isImplemented()` som ligger längst upp i `sortingAlgorithms.c`. Du kan nu köra programmet, om din funktion inte sorterar korrekt kommer programmet att säga till (du kommer få ett meddelande och en assert kommer att aktiveras).

Varje algoritm kommer att sortera tre olika storlekar och tre olika sorteringar (osorterat, sorterat samt bakåtsorterat). Statistik för antalet byten och antalet jämförelser för dessa kommer att skrivas till textfilen result.txt som kommer att ligga i din projektmapp. Behöver du ange hela sökvägen till där filen ska sparas så ändrar du på rad 18 i filen main.c.

Öppna result.txt och bekanta dig med resultatet. Försök relatera sifforna till algoritmernas Ordo (komplexitet). Verkar det stämma med det vi pratat om i kursen? Vilka algoritmer fungerar bra för de olika sorteringarna?